Software Design Specification (SDS)

for

<Detect Spam Mail and Subscription>

Version <1.0>

Prepared by

Group Name: *<Maily Detector>*

<Apichat Laohawan>	**<6622770491>**	**<6622770491@g.siit.tu.ac.th>**
<Krit Meesiri>	**<6622780037>**	**<6622780037@g.siit.tu.ac.th>**
<Apiwit Pengyuak>	**<6622781936>**	**<6622781936@g.siit.tu.ac.th>**
<Pearanat Pipathanasiri>	**<6622782009>**	**<6622782009@g.siit.tu.ac.th>**
<Panod Manodee>	**<6622782033>**	**<6622782033@g.siit.tu.ac.th>**

Instructor: *<Aj.Sasiporn Usanavasin>*

Course: **<CSS323 Software Engineering>**

Date: **<26 April 2026>**

Table of Contents

Table of Contents ii
Revision History iii
Approved By iii
1. Introduction 1
1.1 Purpose 1
1.2 System Overview 1
1.3 Definitions and Acronyms 1
2. Design Considerations 1
2.1 Assumptions 1
2.2 Constraints 2
2.3 System Environment 2
3. Architecture 2
4. Database Schema 3
4.1 Tables, Fields and Relationships 3
4.2 Data Migration 4
5. UX/UI Design 4
5.1 View / Model Element 1…N 4
Appendix A: Project Timeline 5

Revision History

Version	Name	Reason For Changes	Date
1.0	*Apichat Laohawan*	*AI Training*	*10/4/2026*

1. Introduction

1.1 Purpose

This document describes the design of the Spam Mail Detector system, which consists of two components: an AI model for classifying emails, and a Chrome Extension that allows users to invoke that model directly from Gmail.

1.2 System Overview

The system classifies emails into 3 categories:

- **NOT SPAM** — legitimate personal or business email
- **SPAM** — unsolicited, deceptive, or malicious email
- **SUBSCRIPTION** — automated emails from billing, newsletter, or subscription services

The system is divided into 2 layers:

- **AI/ML Layer** — DistilBERT fine-tuned with LoRA, deployed on Hugging Face Spaces (meowslouise/Spammail-detector) via a Gradio web application
- **Client Layer** — A Chrome Extension (Manifest V3) that extracts email text from the Gmail DOM, sends it to the Gradio inference API over HTTPS, and displays the classification result in the extension popup

1.3 Definitions, Acronyms and Abbreviations

TO DO: Please provide a list of all abbreviations and acronyms used in this document sorted in alphabetical order.>

Term	Definition
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
Chrome MV3	Chrome Extension Manifest Version 3
DOM	Document Object Model
F1 Score	Harmonic mean of Precision and Recall;

	primary training metric
GPU	Graphics Processing Unit
Gradio	Open-source Python library for building ML web UIs
HF	Hugging Face — AI platform hosting the model and Space
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
LoRA	Low-Rank Adaptation — parameter-efficient fine-tuning technique
ML	Machine Learning
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
SDS	Software Design Specification
Space	Hugging Face Spaces — cloud platform hosting the Gradio app
SSE	Server-Sent Events — streaming protocol used by Gradio 4.x API
Tokenizer	Component that converts raw text into token IDs for the model

2. Design Considerations

2.1 Assumptions

- The model was trained on **English-language data only** — results for non-English emails (e.g., Thai) may be unreliable
- The model processes **plain text only** — image-based content and attachments are not supported
- The user accesses email via **Gmail (mail.google.com)** on Google Chrome
- The Hugging Face Space is **running and awake** at the time of use — free-tier Spaces may sleep after inactivity

- Input text is **truncated to 256 tokens** at inference time, consistent with training configuration

2.2 Technology Constraints

- **Chrome MV3** — content scripts and popup cannot share memory directly; communication must go through `chrome.runtime.sendMessage`; external API origins must be declared in `host_permissions`
- **DistilBERT token limit** — maximum 512 tokens; model was trained with `max_length=256`, so only the first ~200–250 words of an email are evaluated
- **Hugging Face free tier** — CPU-only inference (no GPU), Space may sleep after ~30 minutes of inactivity, limited concurrency
- **Gradio API versioning** — Gradio 3.x uses a single POST to `/run/predict`; Gradio 4.x uses a two-step flow (`/gradio_api/call/predict` → SSE stream); the extension handles both

2.3 System Environment

Node 1: User Machine (Chrome Browser)

- Gmail running at mail.google.com
- Chrome Extension (MV3)
  - `content.js` — injected into Gmail, extracts email text from DOM
  - `popup.js` — sends text to HF Space API, renders result

Node 2: Hugging Face Spaces (Cloud)

- Gradio App (`meowslouise/Spammail-detector`)
  - Gradio REST API (`/gradio_api/call/predict`)
  - DistilBERT + LoRA inference engine (3-class classifier)
  - Model weights stored in HF Hub

3. Architecture

3.1 Overview

The system is decomposed into two independent layers that communicate exclusively over HTTPS. The decomposition was chosen for the following reasons:

- **Separation of concerns** — the AI inference logic lives entirely in the cloud (HF Space), while the browser-side layer handles only DOM extraction and UI rendering. This means the model can be updated or retrained without touching the extension code, and vice versa.

- **No local model required** — running DistilBERT in-browser would require WebAssembly or ONNX runtime and would result in slow cold-start times. Offloading inference to HF Space keeps the extension lightweight.

The two main components are:

- **Chrome Extension** — responsible for reading email content from Gmail and presenting results to the user
- **Gradio Inference Service** — responsible for running the classification model and returning a label with confidence scores

3.2 Subsystem, Component, or Module 1 …N

Chrome Extension

The extension is built under Chrome Manifest V3 and consists of two active scripts:

content.js

- Injected automatically into every `mail.google.com` page
- Listens for a `getEmailText` message from popup.js
- Queries the Gmail DOM using known CSS selectors (`.a3s.aiL`, `.ii.gt .a3s`, etc.) to locate the open email's subject and body
- Returns the extracted plain text (truncated to 2,000 characters) back to popup.js via `chrome.tabs.sendMessage`

popup.html / popup.js

- Rendered when the user clicks the extension icon
- Contains the Scan button and result display area
- On button click: sends a message to content.js → receives email text → POSTs to Gradio API → parses response → renders label and confidence score
- Implements Thai-language detection: if more than 30% of characters fall in the Thai Unicode range (U+0E00–U+0E7F), a warning is shown before scanning

Gradio Inference Service

Hosted at `meowslouise-spammail-detector.hf.space`, this component wraps the trained model in a Gradio web application. It exposes a REST-compatible inference endpoint that accepts raw email text and returns a classification result.

The underlying model pipeline:

- **Tokenizer** — `distilbert-base-uncased` tokenizer, `max_length=256`, right-padding
- **Model** — DistilBERT for sequence classification, fine-tuned with LoRA adapters (`r=8`, `lora_alpha=32`, target modules: `q_lin`, `v_lin`)

- **Output** — one of three labels: `NOT SPAM`, `SPAM`, `SUBSCRIPTION`, with confidence scores per class

4. Database Schema

4.1 Tables, Fields and Relationships

Provide a description of any new tables, fields and relationships that need to be created for the design.

4.1.1 Database Design

Describe your database design

4.1.2 Database Model

ERD

4.1.3 Table Details

List any new tables that will be needed, for each one including table name, table description, and related tables.

Table Name	Field Name	Data Type	Allow Nulls	Field Description
BinderRequest	SellingRep	Varchar(50)		Get this field from BankPlan.ProposalRepCode But use actual name (Roy Pinnell)
BinderRequest	SigningRep	Varchar(50)		Get this field from Bankplan.SigningRepCode But use actual name (Roy Pinnell)
ProcessBP	ParentProcessID	Int	Yes	This will tie a subprocess to a process
BinderRequest	ProjectedWireDate	Date		The earliest Policy.ProposalWireDate of all included policies in the scenario.

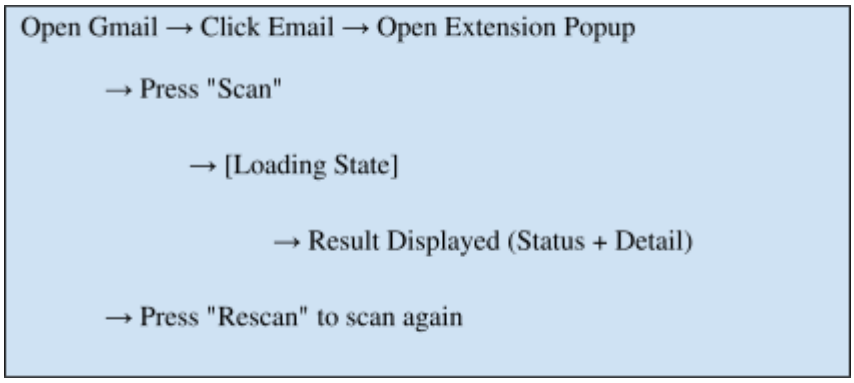
4.2 Data Migration

(Optional) - Provide a description of how existing data should be migrated to new tables and fields.

5. UX/UI Design

5.1 System Navigation Structure

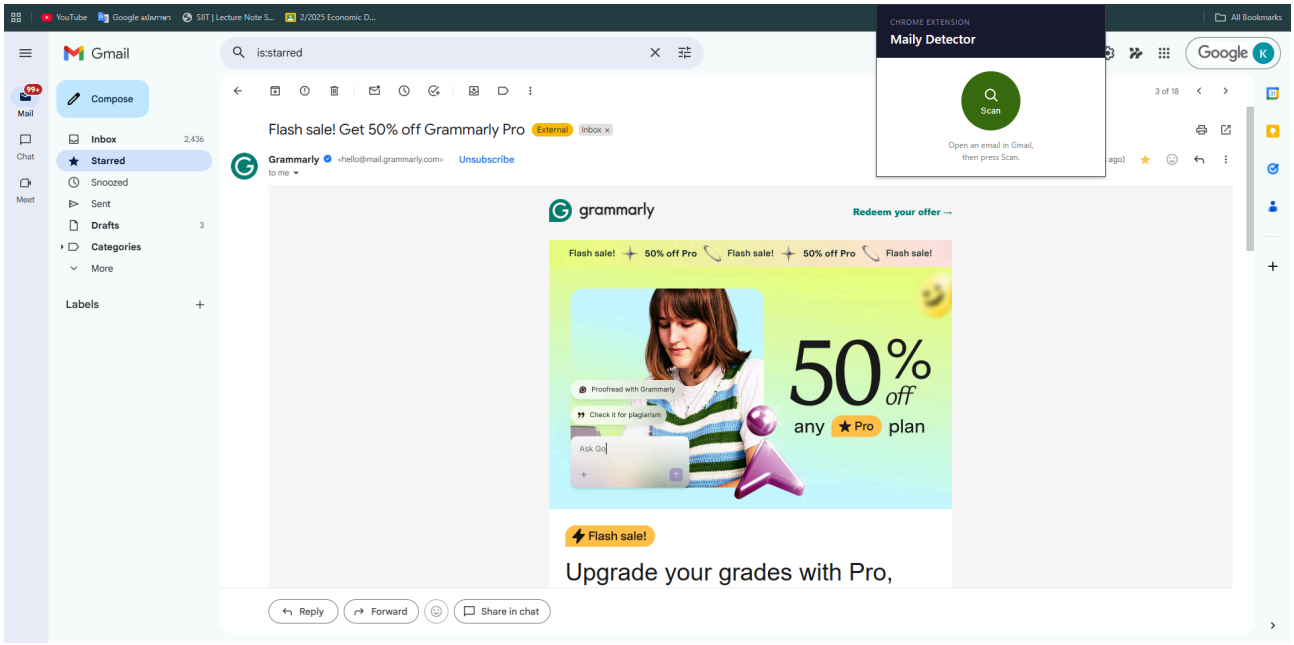
The extension has a flat, single-screen navigation structure — there are no multiple pages or routing involved. The user flow is linear:



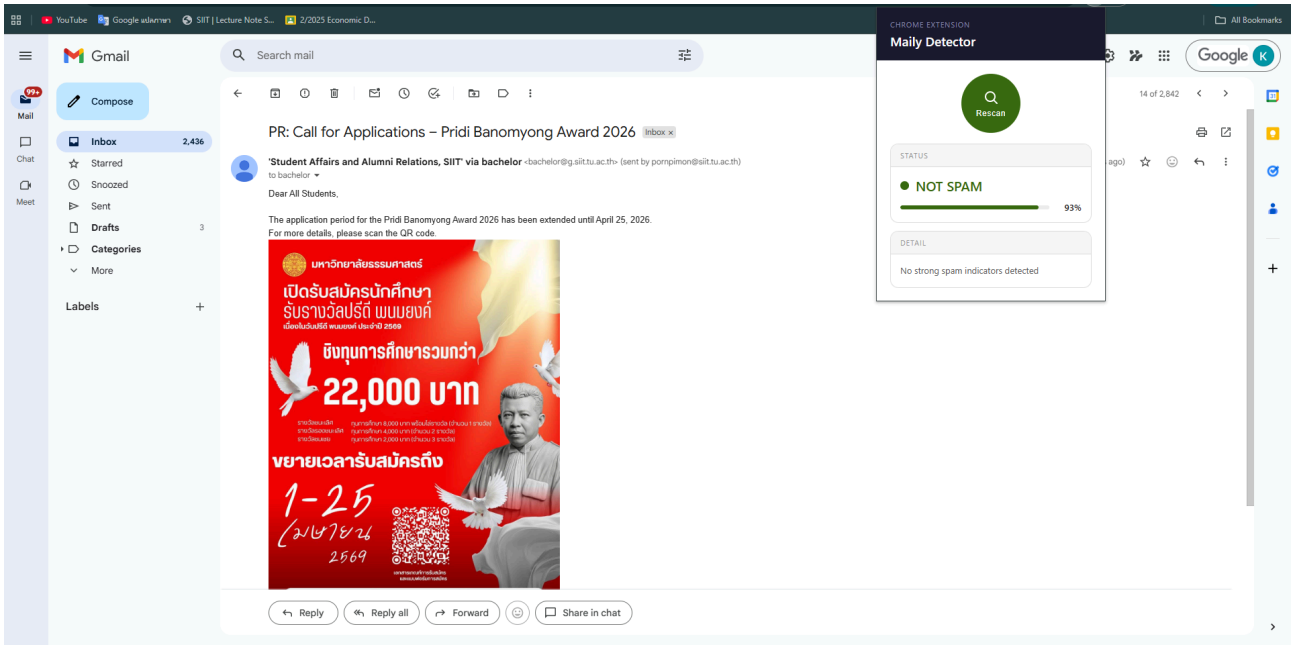
The design intentionally follows **Hick's Law** — only one primary action (Scan) is presented to the user at any time, minimizing decision time and cognitive load.

5.2 UI Design (Screen Design)

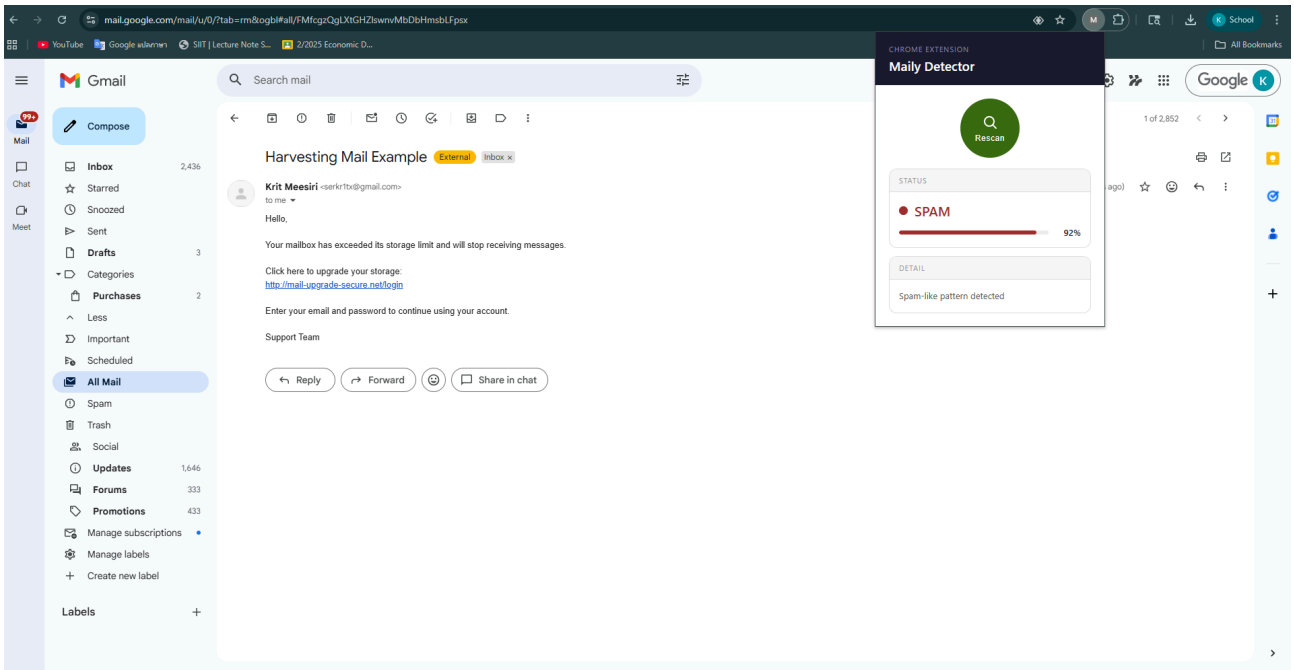
Screen 1: Initial state — extension popup before scanning

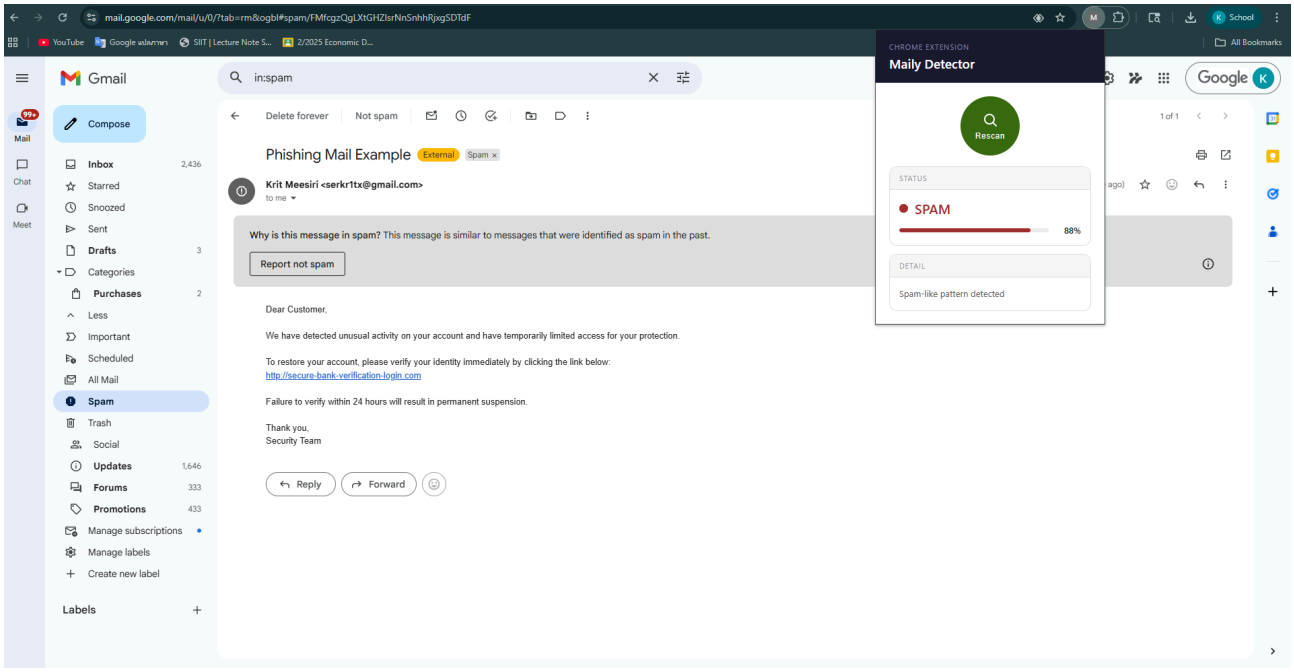


Screen 2: Result state (NOT SPAM) — shows green status, confidence bar, and reason

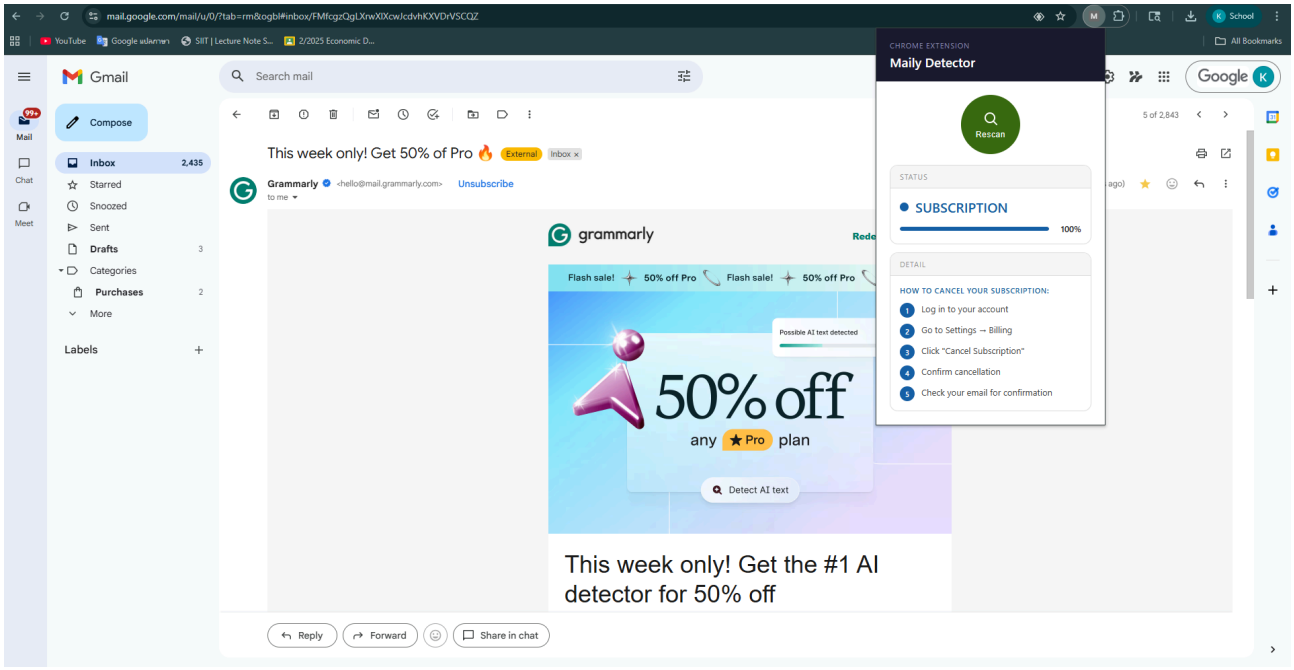


Screen 3: Result state (SPAM) — shows red status with spam keywords detected





Screen 4: Result state (SUBSCRIPTION) — shows blue status with cancel steps



Screen 5: Warning state — Thai language detected

